DSAI 4103 — Business Analytics Course Project

End-to-End Business Analytics & Machine Learning Pipeline

Business Problem: Predicting Airline Active Status

Dataset: Airlines, Airports & Routes (Kaggle)
Objective: Predict whether an airline is currently active (Y) or inactive (N) using features derived from airline data and route/airport statistics.
Business Value: Identifying factors that predict airline survival helps investors, airports, and aviation regulators make data-driven decisions about route planning, infrastructure investment, and market analysis.

---

0. Environment Setup

Run this cell once to install all required packages.

```
# Install required packages (run once)
!pip install scikit-learn shap imbalanced-learn xgboost joblib pandas numpy matplotlib seaborn
```

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.51.0)
Requirement already satisfied: imbalanced-learn in /usr/local/lib/python3.12/dist-packages (0.14.1)
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packages (3.2.0)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (1.5.3)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.12/dist-packages (from shap) (4.67.3)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (from shap) (26.0)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: llvmlite in /usr/local/lib/python3.12/dist-packages (from shap) (0.43.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.2)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from shap) (4.15.0)
Requirement already satisfied: sklearn-compat<0.2,>=0.1.5 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (0.
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.29.7)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0
```

1. Imports & Configuration

> All required libraries are loaded here in one place. Centralising imports keeps the notebook clean and ensures every section below can run without repeating import statements.

```
# ── Core libraries ──────────────────────────────────────────────
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns
import warnings
import os
import joblib
import json
```

```
# ── Machine Learning ──────────────────────────────────────────────────
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    classification_report, confusion_matrix, roc_auc_score,
    ConfusionMatrixDisplay, RocCurveDisplay, accuracy_score,
    precision_score, recall_score, f1_score
)
from sklearn.inspection import permutation_importance
from imblearn.over_sampling import SMOTE

# ── Explainability ────────────────────────────────────────────────────
import shap

# ── Settings ──────────────────────────────────────────────────────────
warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', 50)
pd.set_option('display.float_format', '{:.2f}'.format)
sns.set_theme(style='whitegrid', palette='muted')
RANDOM_STATE = 42

# Output directory for Power BI exports
os.makedirs('powerbi_exports', exist_ok=True)
print(' All libraries loaded successfully.')
```

```
 All libraries loaded successfully.
```

---

2. Data Acquisition & Description

> We load the three raw source files — **Airlines**, **Airports**, and **Routes** — and inspect their structure. This step confirms the data loaded correctly and gives an early view of column names, data types, and the scale of each dataset before any transformation.

```
# Airlines
airlines = pd.read_csv('airlines.csv')

# Airports
airports = pd.read_csv('airports.csv')

# Routes
routes = pd.read_csv('routes.csv')

# Check
print("Dataset shapes:")
print("airlines:", airlines.shape)
print("airports:", airports.shape)
print("routes:", routes.shape)

print("\nColumns:")
print("Airlines:", airlines.columns.tolist())
print("Airports:", airports.columns.tolist())
print("Routes:", routes.columns.tolist())
```

```
Dataset shapes:
airlines: (6161, 6)
airports: (7698, 11)
routes: (67663, 9)

Columns:
Airlines: ['Name', 'IATA', 'ICAO', 'Callsign', 'Country', 'Active']
Airports: ['Name', 'City', 'Country', 'IATA', 'ICAO', 'Latitude', 'Longitude', 'Altitude', 'Timezone', 'DST', 'Timezone.1']
Routes: ['Airline', 'Airline ID', 'Source Airport', 'Source Airport ID', 'Destination Airport', 'Destination Airport ID', 'Codes
```

```
# ── Airlines overview ─────────────────────────────────────────────────
print('=== AIRLINES ===')
display(airlines.head())
print('\nData types & nulls:')
display(airlines.info())
```

```
=== AIRLINES ===
```

	Name	IATA	ICAO	Callsign	Country	Active
0	Private flight	-	NaN	NaN	NaN	Y
1	135 Airways	NaN	GNL	GENERAL	United States	N
2	1Time Airline	1T	RNX	NEXTIME	South Africa	Y
3	2 Sqn No 1 Elementary Flying Training School	NaN	WYT	NaN	United Kingdom	N
4	213 Flight Unit	NaN	TFU	NaN	Russia	N

```
Data types & nulls:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6161 entries, 0 to 6160
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Name      6161 non-null   object
 1   IATA      1534 non-null   object
 2   ICAO      6075 non-null   object
 3   Callsign  5353 non-null   object
 4   Country   6146 non-null   object
 5   Active    6161 non-null   object
dtypes: object(6)
memory usage: 288.9+ KB
None
```

```
# ── Airports overview ──────────────────────────────────────────────────
print('=== AIRPORTS ===')
display(airports.head())
display(airports.describe())
```

```
=== AIRPORTS ===
```

	Name	City	Country	IATA	ICAO	Latitude	Longitude	Altitude	Timezone	DST	Timezone.1
0	Goroka Airport	Goroka	Papua New Guinea	GKA	AYGA	-6.08	145.39	5282	10	U	Pacific/Port_Moresby
1	Madang Airport	Madang	Papua New Guinea	MAG	AYMD	-5.21	145.79	20	10	U	Pacific/Port_Moresby
2	Mount Hagen Kagamuga Airport	Mount Hagen	Papua New Guinea	HGU	AYMH	-5.83	144.30	5388	10	U	Pacific/Port_Moresby
3	Nadzab Airport	Nadzab	Papua New Guinea	LAE	AYNZ	-6.57	146.73	239	10	U	Pacific/Port_Moresby
4	Port Moresby Jacksons International Airport	Port Moresby	Papua New Guinea	POM	AYPY	-9.44	147.22	146	10	U	Pacific/Port_Moresby

	Latitude	Longitude	Altitude
count	7698.00	7698.00	7698.00
mean	25.81	-1.39	1015.87
std	28.40	86.52	1628.78
min	-90.00	-179.88	-1266.00
25%	6.91	-78.97	63.00

```
# ── Routes overview ────────────────────────────────────────────────────
print('=== ROUTES ===')
display(routes.head())
display(routes.describe())
```

```
=== ROUTES ===
```

	Airline	Airline ID	Source Airport	Source Airport ID	Destination Airport	Destination Airport ID	Codeshare	Stops	Equipment
0	2B	410	AER	2965	KZN	2990	NaN	0	CR2
1	2B	410	ASF	2966	KZN	2990	NaN	0	CR2
2	2B	410	ASF	2966	MRV	2962	NaN	0	CR2
3	2B	410	CEK	2968	KZN	2990	NaN	0	CR2
4	2B	410	CEK	2968	OVB	4078	NaN	0	CR2

	Stops
count	67663.00
mean	0.00
std	0.01
min	0.00
25%	0.00
50%	0.00
75%	0.00

3. Data Cleaning & Preprocessing

> Raw data is rarely analysis-ready. Here we handle missing values, standardise column formats, fix inconsistent encodings, and create the binary target variable (`Active_binary`). Clean data is essential — garbage in, garbage out.

```
# --- Airlines ---
airlines['Country'] = airlines['Country'].fillna('').astype(str).str.strip().str.title()

airlines['Active'] = airlines['Active'].fillna('N').astype(str).str.strip().str.upper()
airlines['Active'] = airlines['Active'].apply(lambda x: 'Y' if x == 'Y' else 'N')

airlines['Active_binary'] = (airlines['Active'] == 'Y').astype(int)

# --- Airports ---
airports['Country'] = airports['Country'].fillna('').astype(str).str.strip().str.title()

# --- Routes ---
routes['Stops'] = pd.to_numeric(routes['Stops'], errors='coerce').fillna(0).astype(int)

routes['Codeshare'] = routes['Codeshare'].fillna('N').astype(str).str.strip().str.upper()
routes['Codeshare'] = routes['Codeshare'].apply(lambda x: 'Y' if x == 'Y' else 'N')

print("Cleaning done ")
print("Airlines:", airlines.shape)
print("Routes:", routes.shape)
print("\nActive counts:")
print(airlines['Active'].value_counts())
```

```
Cleaning done
Airlines: (6161, 7)
Routes: (67663, 9)

Active counts:
Active
N    4907
Y    1254
Name: count, dtype: int64
```

```
# ── Missing values summary ──────────────────────────────────────────
def missing_summary(df, name):
    miss = df.isnull().sum()
    miss = miss[miss > 0].reset_index()
    miss.columns = ['Column', 'Missing_Count']
    miss['Missing_%'] = (miss['Missing_Count'] / len(df) * 100).round(2)
    print(f'\n=== {name} — Missing Values ===')
```

```
    if len(miss) == 0:
        print('  No missing values.')
    else:
        display(miss)

missing_summary(airlines, 'Airlines')
missing_summary(airports, 'Airports')
missing_summary(routes, 'Routes')
```

```
=== Airlines — Missing Values ===
```

	Column	Missing_Count	Missing_%
0	IATA	4627	75.10
1	ICAO	86	1.40
2	Callsign	808	13.11

```
=== Airports — Missing Values ===
```

	Column	Missing_Count	Missing_%
0	City	49	0.64

```
=== Routes — Missing Values ===
```

	Column	Missing_Count	Missing_%
0	Equipment	18	0.03

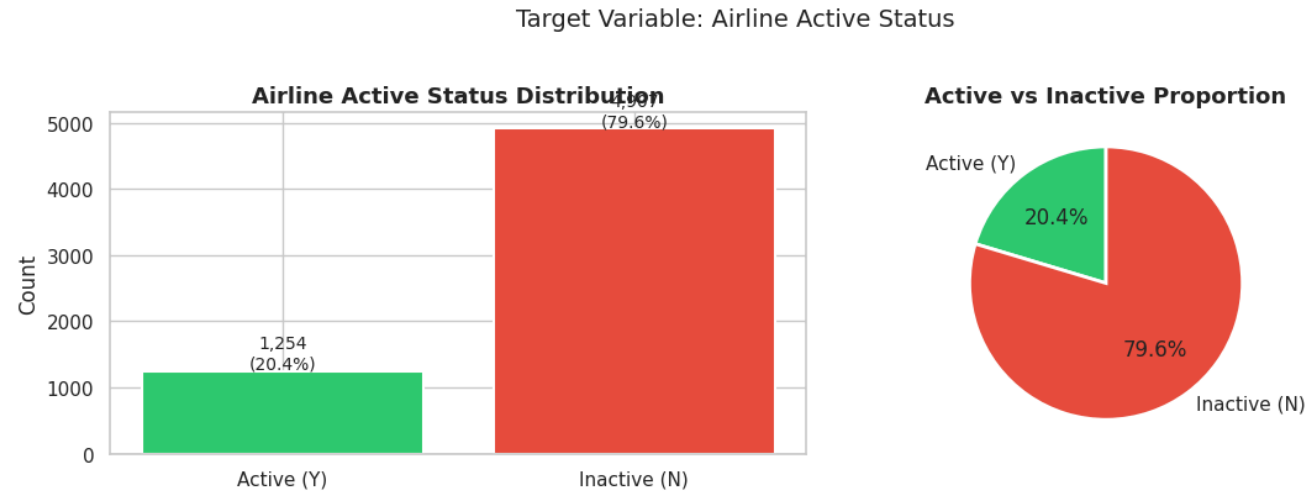
---

4. Exploratory Data Analysis (EDA)

> Before building any model we explore the data visually and statistically. EDA reveals the severe class imbalance between active and inactive airlines, shows which countries dominate, and uncovers route-activity patterns that directly inform our feature engineering choices.

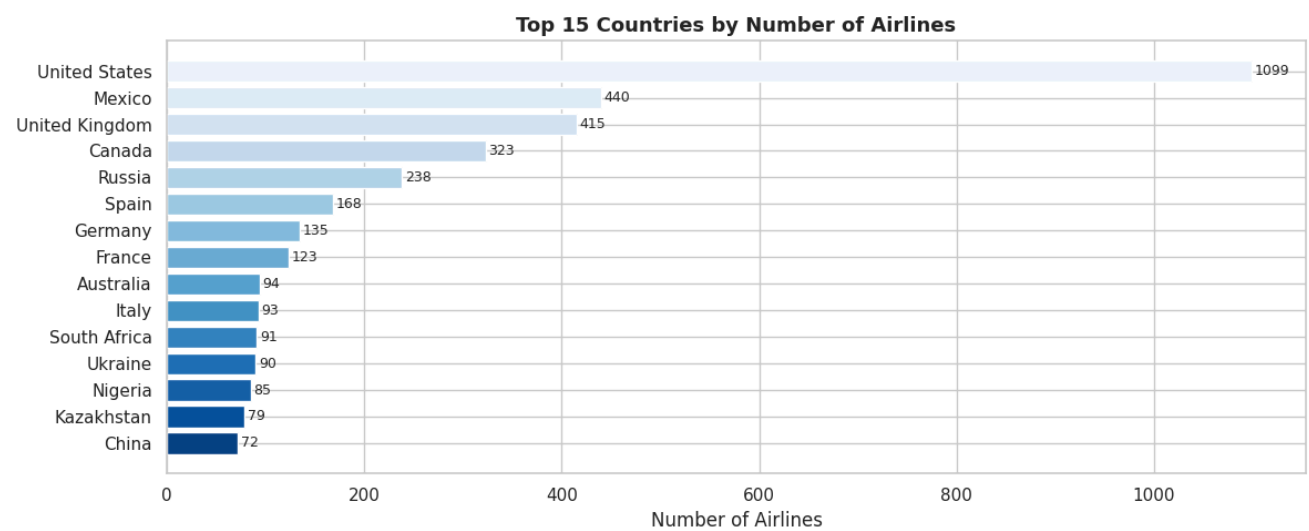
```
# ── 4.1 Target Distribution ──────────────────────────────────────────────────
# IMPORTANT: Explicitly compute counts per label to avoid ordering issues
y_count = (airlines['Active'] == 'Y').sum()   # Active airlines
n_count = (airlines['Active'] == 'N').sum()   # Inactive airlines
total   = len(airlines)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Bar chart — explicit labels so order never gets swapped
bar_labels = ['Active (Y)', 'Inactive (N)']
bar_values = [y_count, n_count]
bar_colors = ['#2ecc71', '#e74c3c']

bars = axes[0].bar(bar_labels, bar_values, color=bar_colors,
                   edgecolor='white', linewidth=1.5)
axes[0].set_title('Airline Active Status Distribution', fontsize=13, fontweight='bold')
axes[0].set_ylabel('Count')
for bar, val in zip(bars, bar_values):
    axes[0].text(bar.get_x() + bar.get_width()/2, val + 40,
                 f'{val:,}\n({val/total*100:.1f}%)',
                 ha='center', fontsize=10)

# Pie chart — same explicit order
axes[1].pie(
    [y_count, n_count],
    labels=['Active (Y)', 'Inactive (N)'],
    autopct='%1.1f%%',
    colors=['#2ecc71', '#e74c3c'],
    startangle=90,
    wedgeprops={'edgecolor': 'white', 'linewidth': 2}
)
axes[1].set_title('Active vs Inactive Proportion', fontsize=13, fontweight='bold')

plt.suptitle('Target Variable: Airline Active Status', fontsize=14, y=1.02)
plt.tight_layout()
plt.savefig('powerbi_exports/fig_target_distribution.png', dpi=150, bbox_inches='tight')
```

```
plt.show()

print(f'Active   (Y): {y_count:,}  ({y_count/total*100:.1f}%)')
print(f'Inactive (N): {n_count:,}  ({n_count/total*100:.1f}%)')
```



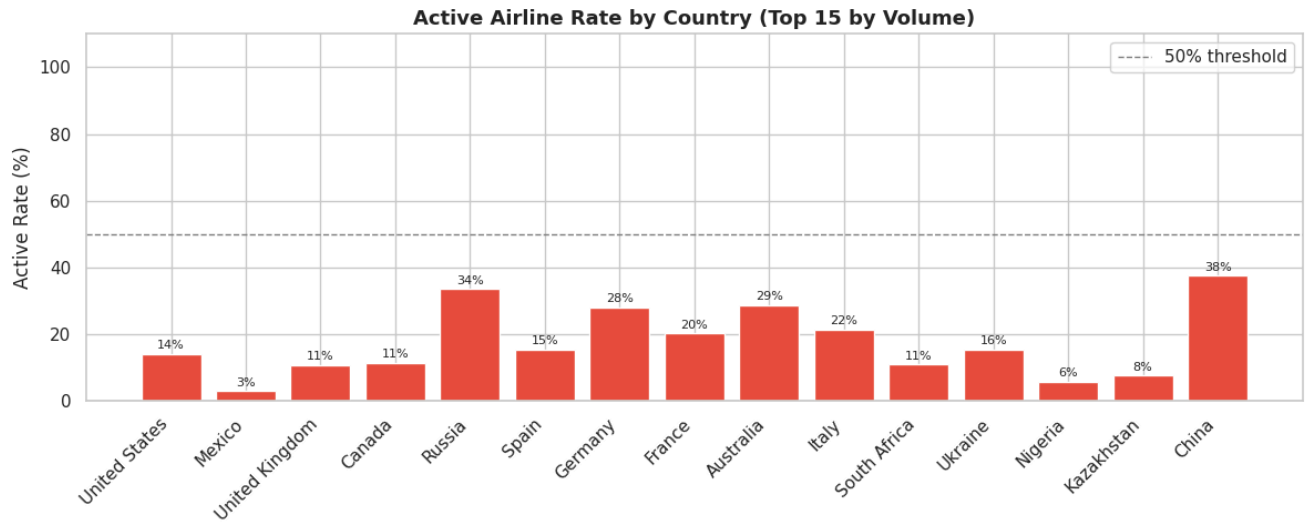

```
Active   (Y): 1,254  (20.4%)
Inactive (N): 4,907  (79.6%)
```

```
# ── 4.2 Top countries by airline count ──────────────────────────────────
top_countries = (airlines.groupby('Country').size()
                 .sort_values(ascending=False)
                 .head(15))

plt.figure(figsize=(12, 5))
bars = plt.barh(top_countries.index[::-1], top_countries.values[::-1],
                color=sns.color_palette('Blues_r', 15))
plt.title('Top 15 Countries by Number of Airlines', fontsize=13, fontweight='bold')
plt.xlabel('Number of Airlines')
for bar, val in zip(bars, top_countries.values[::-1]):
    plt.text(val + 3, bar.get_y() + bar.get_height()/2,
             str(val), va='center', fontsize=9)
plt.tight_layout()
plt.savefig('powerbi_exports/fig_top_countries.png', dpi=150, bbox_inches='tight')
plt.show()
print(f'Top country: {top_countries.index[0]} with {top_countries.iloc[0]:,} airlines')
```

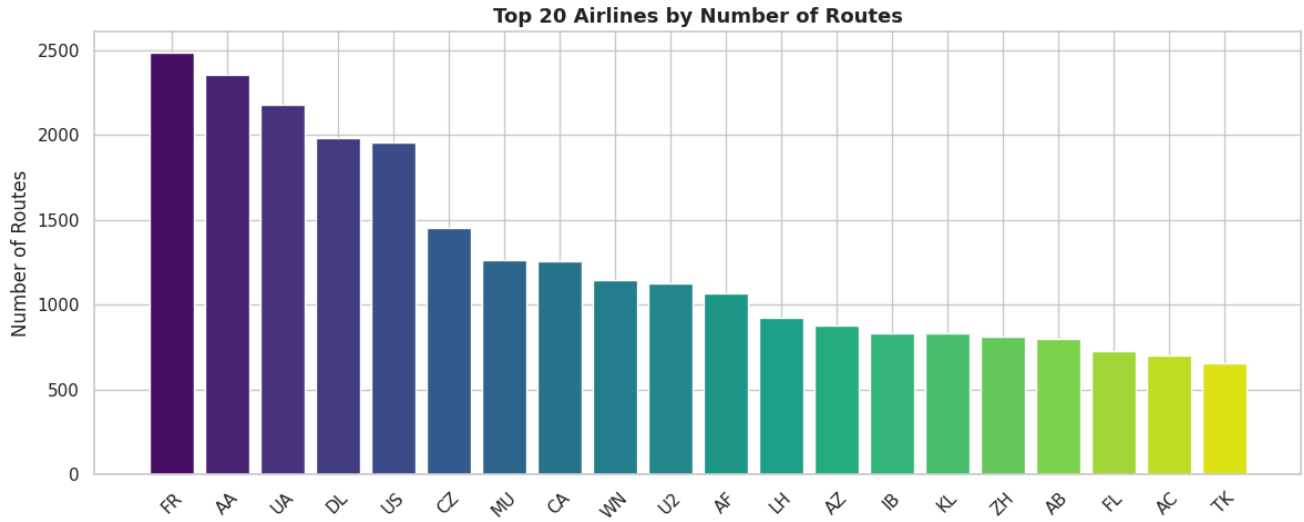



```
Top country: United States with 1,099 airlines
```

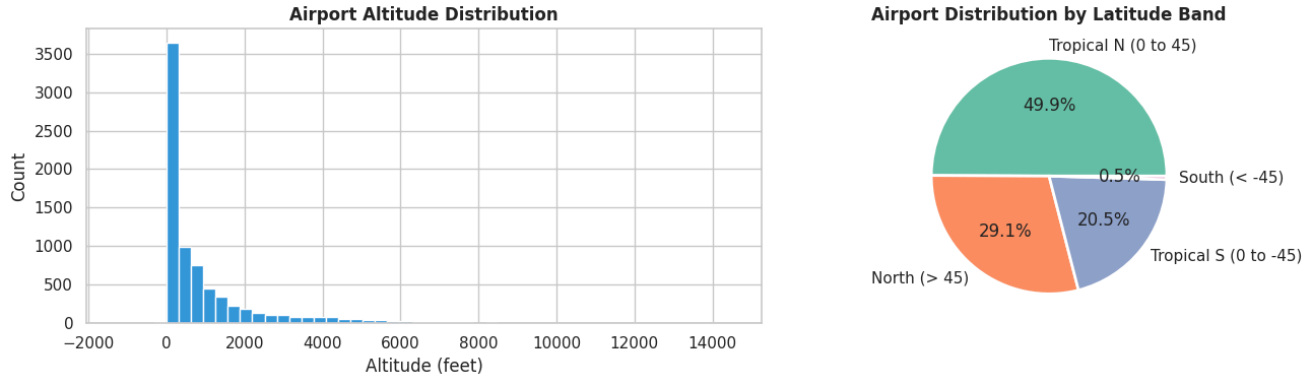
```
# ── 4.3 Active rate by top 15 countries ──────────────────────────────────
country_stats = (airlines.groupby('Country')['Active_binary']
                 .agg(['sum','count','mean'])
                 .rename(columns={'sum':'active','count':'total','mean':'active_rate'})
                 .sort_values('total', ascending=False)
                 .head(15))
country_stats['active_rate_pct'] = country_stats['active_rate'] * 100

fig, ax = plt.subplots(figsize=(12, 5))
colors = ['#2ecc71' if r >= 50 else '#e74c3c' for r in country_stats['active_rate_pct']]
bars = ax.bar(country_stats.index, country_stats['active_rate_pct'], color=colors, edgecolor='white')
ax.axhline(50, color='gray', linestyle='--', linewidth=1, label='50% threshold')
ax.set_title('Active Airline Rate by Country (Top 15 by Volume)', fontsize=13, fontweight='bold')
ax.set_ylabel('Active Rate (%)')
ax.set_ylim(0, 110)
plt.xticks(rotation=45, ha='right')
for bar, pct in zip(bars, country_stats['active_rate_pct']):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1.5,
            f'{pct:.0f}%', ha='center', fontsize=8)
ax.legend()
plt.tight_layout()
plt.savefig('powerbi_exports/fig_active_rate_country.png', dpi=150, bbox_inches='tight')
plt.show()
```




```
# ── 4.4 Routes: busiest airlines ──────────────────────────────────────────
route_counts = routes.groupby('Airline').size().reset_index(name='route_count')
top_airlines_routes = route_counts.sort_values('route_count', ascending=False).head(20)

plt.figure(figsize=(12, 5))
plt.bar(range(len(top_airlines_routes)), top_airlines_routes['route_count'],
        color=sns.color_palette('viridis', 20))
plt.xticks(range(len(top_airlines_routes)), top_airlines_routes['Airline'], rotation=45)
plt.title('Top 20 Airlines by Number of Routes', fontsize=13, fontweight='bold')
plt.ylabel('Number of Routes')
plt.tight_layout()
plt.savefig('powerbi_exports/fig_top_airlines_routes.png', dpi=150, bbox_inches='tight')
plt.show()
```

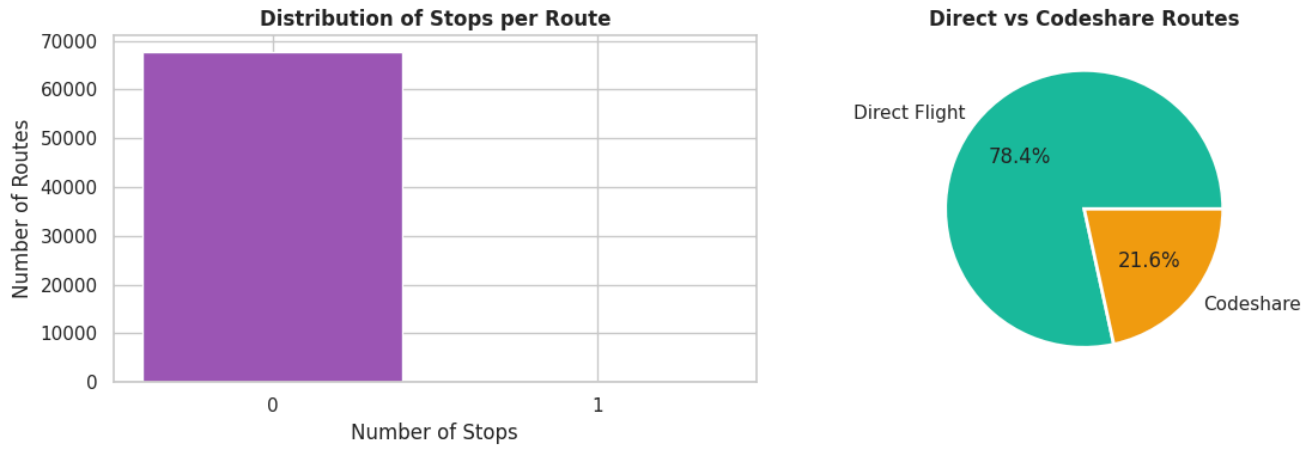



```
# ── 4.5 Airport altitude distribution by timezone ──────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

axes[0].hist(airports['Altitude'].dropna(), bins=50, color='#3498db', edgecolor='white')
axes[0].set_title('Airport Altitude Distribution', fontsize=12, fontweight='bold')
axes[0].set_xlabel('Altitude (feet)')
axes[0].set_ylabel('Count')

lat_bins = pd.cut(airports['Latitude'], bins=[-90, -45, 0, 45, 90],
                  labels=['South (< -45)', 'Tropical S (0 to -45)',
                          'Tropical N (0 to 45)', 'North (> 45)'])
lat_counts = lat_bins.value_counts()
axes[1].pie(lat_counts.values, labels=lat_counts.index, autopct='%1.1f%%',
            colors=sns.color_palette('Set2', 4),
            wedgeprops={'edgecolor': 'white', 'linewidth': 2})
axes[1].set_title('Airport Distribution by Latitude Band', fontsize=12, fontweight='bold')

plt.tight_layout()
plt.savefig('powerbi_exports/fig_airport_distribution.png', dpi=150, bbox_inches='tight')
plt.show()
```




```
# ── 4.6 Routes: stops distribution ─────────────────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

stop_counts = routes['Stops'].value_counts().sort_index()
axes[0].bar(stop_counts.index.astype(str), stop_counts.values, color='#9b59b6', edgecolor='white')
axes[0].set_title('Distribution of Stops per Route', fontsize=12, fontweight='bold')
```

```
axes[0].set_xlabel('Number of Stops')
axes[0].set_ylabel('Number of Routes')

codeshare_counts = routes['Codeshare'].fillna('Direct').value_counts()
axes[1].pie(codeshare_counts.values,
            labels=['Direct Flight', 'Codeshare'],
            autopct='%1.1f%%', colors=['#1abc9c', '#f39c12'],
            wedgeprops={'edgecolor': 'white', 'linewidth': 2})
axes[1].set_title('Direct vs Codeshare Routes', fontsize=12, fontweight='bold')

plt.tight_layout()
plt.savefig('powerbi_exports/fig_routes_analysis.png', dpi=150, bbox_inches='tight')
plt.show()
```




```
# ── 4.7 Statistical Summary ──────────────────────────────────────────────
print('=== AIRLINES STATISTICAL SUMMARY ===')
display(airlines.describe(include='all').T)

print('\n=== AIRPORTS STATISTICAL SUMMARY ===')
display(airports.describe(include='all').T)

# Key business stats
print('\n=== KEY STATISTICS ===')
print(f'Total Airlines  : {len(airlines):,}')
print(f'Active (Y)      : {(airlines["Active"] == "Y").sum():,}  ({(airlines["Active"] == "Y").mean()*100:.1f}%)')
print(f'Inactive (N)    : {(airlines["Active"] == "N").sum():,}  ({(airlines["Active"] == "N").mean()*100:.1f}%)')
print(f'Total Airports  : {len(airports):,}')
print(f'Total Routes    : {len(routes):,}')
print(f'Countries       : {airlines["Country"].nunique():,}')
print(f'Airlines w/ IATA: {airlines["IATA"].notna().sum():,}')
```

```
=== AIRLINES STATISTICAL SUMMARY ===
```

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
Name	6161	6072	Royal Air Force	5	NaN	NaN	NaN	NaN	NaN	NaN	NaN
IATA	1534	1121	1I	7	NaN	NaN	NaN	NaN	NaN	NaN	NaN
ICAO	6075	5854	\N	188	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Callsign	5353	5262	Inc.	20	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Country	6161	277	United States	1099	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Active	6161	2	N	4907	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Active_binary	6161.00	NaN	NaN	NaN	0.20	0.40	0.00	0.00	0.00	0.00	1.00

```
=== AIRPORTS STATISTICAL SUMMARY ===
```

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
Name	7698	7658	Newcastle Airport	3	NaN	NaN	NaN	NaN	NaN	NaN	NaN
City	7649	6955	London	9	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Country	7698	237	United States	1512	NaN	NaN	NaN	NaN	NaN	NaN	NaN
IATA	7698	6073	\N	1626	NaN	NaN	NaN	NaN	NaN	NaN	NaN
ICAO	7698	7698	UKDM	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Latitude	7698.00	NaN	NaN	NaN	25.81	28.40	-90.00	6.91	34.09	47.24	89.50
Longitude	7698.00	NaN	NaN	NaN	-1.39	86.52	-179.88	-78.97	6.38	56.00	179.95
Altitude	7698.00	NaN	NaN	NaN	1015.87	1628.78	-1266.00	63.00	352.00	1203.00	14472.00
Timezone	7698	41	1	1184	NaN	NaN	NaN	NaN	NaN	NaN	NaN
DST	7698	8	U	1862	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Timezone.1	7698	308	\N	1021	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```
=== KEY STATISTICS ===
Total Airlines  : 6,161
Active (Y)      : 1,254  (20.4%)
Inactive (N)    : 4,907  (79.6%)
Total Airports  : 7,698
Total Routes    : 67,663
Countries       : 277
Airlines w/ IATA: 1,534
```

5. Feature Engineering

> Raw columns like airline name or country string are not directly usable by machine learning algorithms. Here we derive 15 meaningful numeric features from route data, identifier presence, and country-level statistics. Strong features are the single biggest driver of model performance.

```
# ── 5.1 Route-level features per airline ──────────────────────────────
# Count routes per airline (by IATA code)
route_agg = routes.groupby('Airline').agg(
    total_routes=('Airline', 'count'),
    unique_src_airports=('Source Airport', 'nunique'),
    unique_dst_airports=('Destination Airport', 'nunique'),
    codeshare_routes=('Codeshare', lambda x: (x == 'Y').sum()),
    avg_stops=('Stops', 'mean'),
    direct_routes=('Stops', lambda x: (x == 0).sum())
).reset_index()

route_agg.rename(columns={'Airline': 'IATA'}, inplace=True)

# Derived ratios
route_agg['codeshare_ratio'] = route_agg['codeshare_routes'] / route_agg['total_routes']
route_agg['direct_ratio'] = route_agg['direct_routes'] / route_agg['total_routes']
route_agg['network_breadth'] = route_agg['unique_src_airports'] + route_agg['unique_dst_airports']

print(f'Route aggregation shape: {route_agg.shape}')
display(route_agg.head())
```

```
Route aggregation shape: (568, 10)
```

	IATA	total_routes	unique_src_airports	unique_dst_airports	codeshare_routes	avg_stops	direct_routes	codeshare_ratio	dir
0	2B	42	17	17	0	0.00	42	0.00	
1	2G	21	11	10	0	0.00	21	0.00	
2	2I	18	8	8	0	0.00	18	0.00	
3	2J	22	9	9	0	0.00	22	0.00	
4	2K	12	6	6	0	0.00	12	0.00	

```
# ── 5.2 Merge datasets ──────────────────────────────────────────────────
df = airlines.merge(route_agg, on='IATA', how='left')

# Fill missing route features with 0 (airlines with no routes in dataset)
route_cols = ['total_routes','unique_src_airports','unique_dst_airports',
              'codeshare_routes','avg_stops','direct_routes',
              'codeshare_ratio','direct_ratio','network_breadth']
df[route_cols] = df[route_cols].fillna(0)

print(f'Merged dataset shape: {df.shape}')
display(df.head())
```

```
Merged dataset shape: (6161, 16)
```

	Name	IATA	ICAO	Callsign	Country	Active	Active_binary	total_routes	unique_src_airports	unique_dst_airports	code
0	Private flight	-	NaN	NaN		Y	1	0.00	0.00	0.00	
1	135 Airways	NaN	GNL	GENERAL	United States	N	0	0.00	0.00	0.00	
2	1Time Airline	1T	RNX	NEXTIME	South Africa	Y	1	0.00	0.00	0.00	
3	2 Sqn No 1 Elementary Flying Training School	NaN	WYT	NaN	United Kingdom	N	0	0.00	0.00	0.00	
4	213 Flight Unit	NaN	TFU	NaN	Russia	N	0	0.00	0.00	0.00	

```
# ── 5.3 Encode categorical features ──────────────────────────────────────
# Has IATA code (many legitimate airlines have one)
df['has_iata']     = (~df['IATA'].isnull()).astype(int)
df['has_icao']     = (~df['ICAO'].isnull()).astype(int)
df['has_callsign'] = (~df['Callsign'].isnull()).astype(int)

# Country-level active rate (target encoding)
country_active_rate = df.groupby('Country')['Active_binary'].mean().to_dict()
df['country_active_rate'] = df['Country'].map(country_active_rate).fillna(df['Active_binary'].mean())

# Has routes at all
df['has_routes'] = (df['total_routes'] > 0).astype(int)

# Route size bucket
df['route_size_bucket'] = pd.cut(
    df['total_routes'],
    bins=[-1, 0, 10, 50, 200, 10000],
    labels=[0, 1, 2, 3, 4]
).astype(float).fillna(0).astype(int)

print('Feature engineering complete.')
print(f'Total features created: {df.shape[1]}')
```

```
Feature engineering complete.
Total features created: 22
```

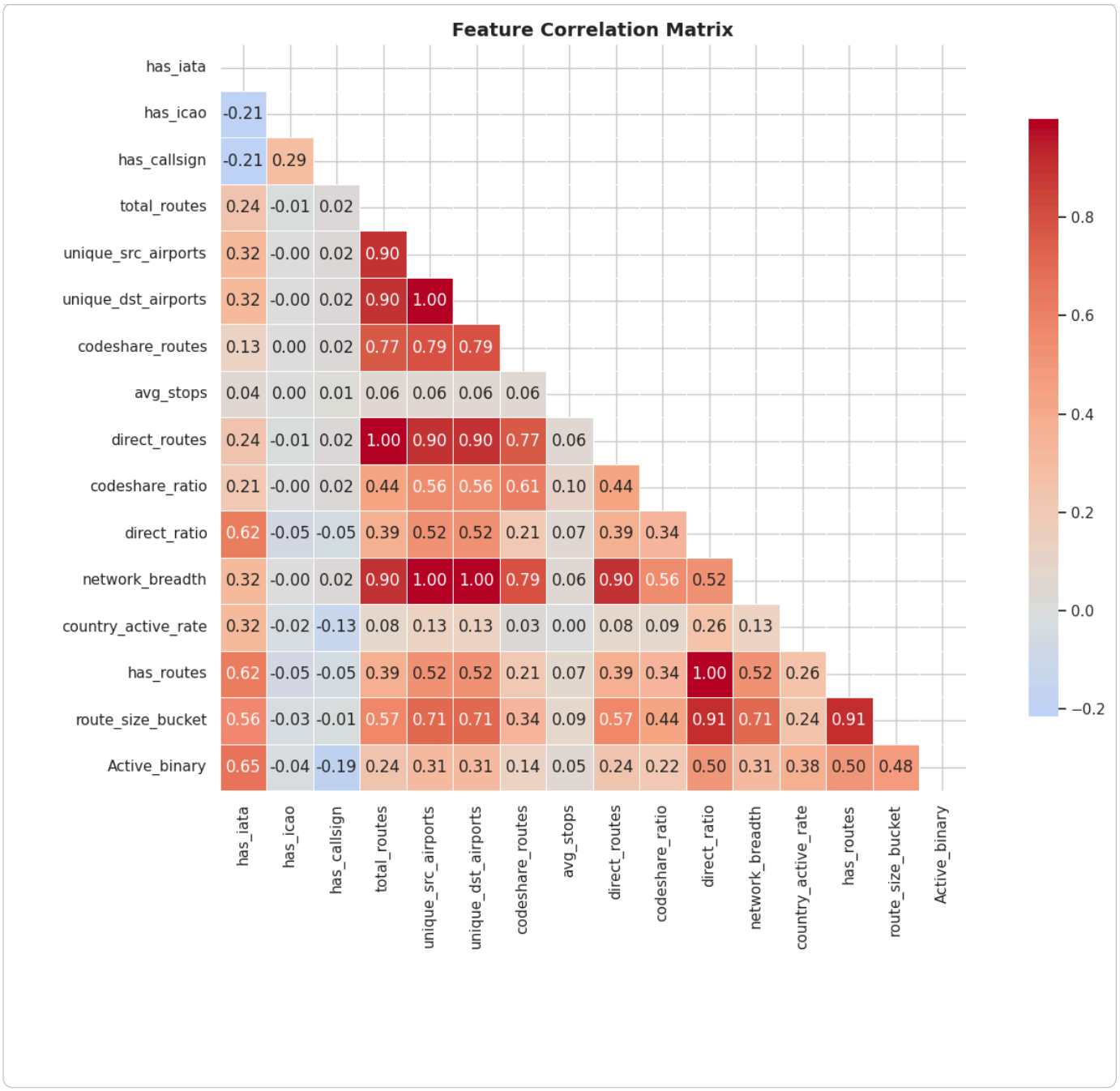
```
FEATURES = [
    'has_iata', 'has_icao', 'has_callsign',
    'total_routes', 'unique_src_airports', 'unique_dst_airports',
    'codeshare_routes', 'avg_stops', 'direct_routes',
    'codeshare_ratio', 'direct_ratio', 'network_breadth',
```

```
    'country_active_rate', 'has_routes', 'route_size_bucket'
]

TARGET = 'Active_binary'

X = df[FEATURES].copy()
y = df[TARGET].copy()

print(f'Feature matrix: {X.shape}')
print(f'Target distribution: {y.value_counts().to_dict()}')
```

```
Feature matrix: (6161, 15)
Target distribution: {0: 4907, 1: 1254}
```

```
os.makedirs('powerbi_exports', exist_ok=True)
```

```
# ── 5.5 Correlation heatmap ──────────────────────────────────────────────────
corr_data = X.copy()
corr_data['Active_binary'] = y
corr_matrix = corr_data.corr()

plt.figure(figsize=(14, 10))
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
sns.heatmap(corr_matrix, mask=mask, annot=True, fmt='.2f',
            cmap='coolwarm', center=0, square=True,
            linewidths=0.5, cbar_kws={'shrink': 0.8})
plt.title('Feature Correlation Matrix', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('powerbi_exports/fig_correlation_heatmap.png', dpi=150, bbox_inches='tight')
plt.show()
```



6. Model Development (Ensemble / AutoML-style)

We build three classifiers and combine them in a **Voting Ensemble** for robust predictions.

```
# ── 6.1 Train/test split ──────────────────────────────────────────────
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)

print(f'Training set: {X_train.shape[0]:,} samples')
print(f'Test set    : {X_test.shape[0]:,} samples')
print(f'Class balance (train): {y_train.mean()*100:.1f}% active')
```

```
Training set: 4,928 samples
Test set    : 1,233 samples
Class balance (train): 20.4% active
```

```
# ── 6.2 Handle class imbalance with SMOTE ──────────────────────────────
smote = SMOTE(random_state=RANDOM_STATE)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)
```

```
print('After SMOTE resampling:')
print(f'  Samples: {X_train_res.shape[0]:,}')
print(f'  Class balance: {y_train_res.mean()*100:.1f}% active')
```

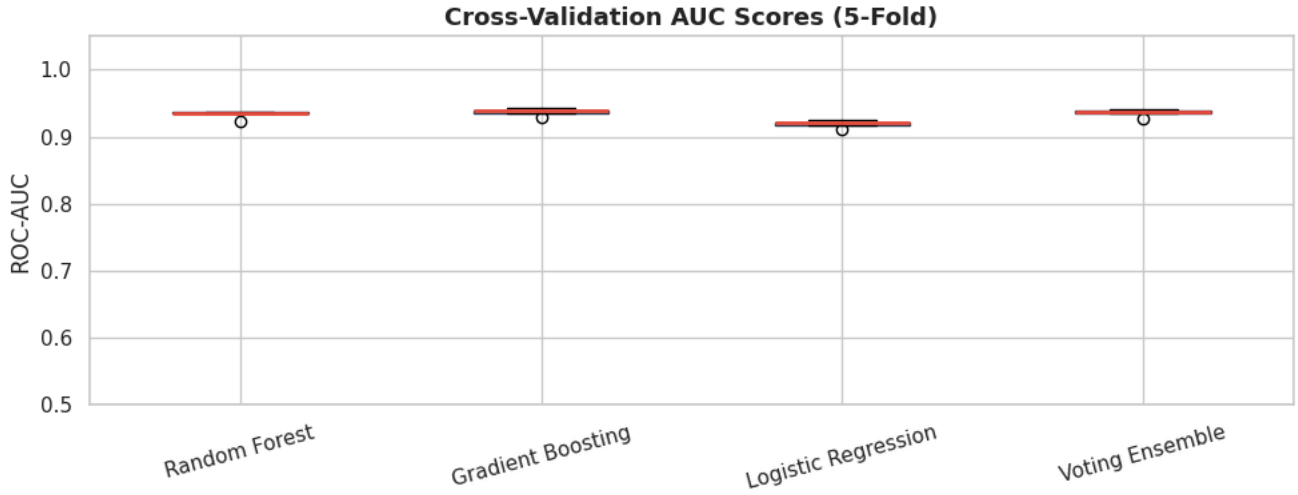
```
After SMOTE resampling:
  Samples: 7,850
  Class balance: 50.0% active
```

```
# ── 6.3 Define base models ──────────────────────────────────────────────
rf_model = RandomForestClassifier(
    n_estimators=200, max_depth=12,
    min_samples_leaf=5, class_weight='balanced',
    random_state=RANDOM_STATE, n_jobs=-1
)

gb_model = GradientBoostingClassifier(
    n_estimators=200, learning_rate=0.05,
    max_depth=5, random_state=RANDOM_STATE
)

lr_model = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', LogisticRegression(max_iter=1000, class_weight='balanced',
                               C=1.0, random_state=RANDOM_STATE))
])

# ── Voting Ensemble (soft voting averages probabilities) ────────────────
ensemble = VotingClassifier(
    estimators=[('rf', rf_model), ('gb', gb_model), ('lr', lr_model)],
    voting='soft',
    weights=[3, 2, 1]  # RF weighted highest
)

print('Models defined.')
```

```
Models defined.
```

```
# ── 6.4 Cross-validation scores ─────────────────────────────────────────
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

models = [('Random Forest', rf_model), ('Gradient Boosting', gb_model),
          ('Logistic Regression', lr_model), ('Voting Ensemble', ensemble)]

cv_results = {}
for name, model in models:
    scores = cross_val_score(model, X_train_res, y_train_res,
                             cv=cv, scoring='roc_auc', n_jobs=-1)
    cv_results[name] = scores
    print(f'{name:<25} AUC: {scores.mean():.4f} ± {scores.std():.4f}')

# Plot CV results
plt.figure(figsize=(10, 4))
plt.boxplot([cv_results[m] for m in cv_results],
            labels=list(cv_results.keys()), patch_artist=True,
            boxprops=dict(facecolor='#3498db', color='#2c3e50'),
            medianprops=dict(color='#e74c3c', linewidth=2))
plt.title('Cross-Validation AUC Scores (5-Fold)', fontsize=13, fontweight='bold')
plt.ylabel('ROC-AUC')
plt.ylim(0.5, 1.05)
plt.xticks(rotation=15)
plt.tight_layout()
plt.savefig('powerbi_exports/fig_cv_scores.png', dpi=150, bbox_inches='tight')
plt.show()
```

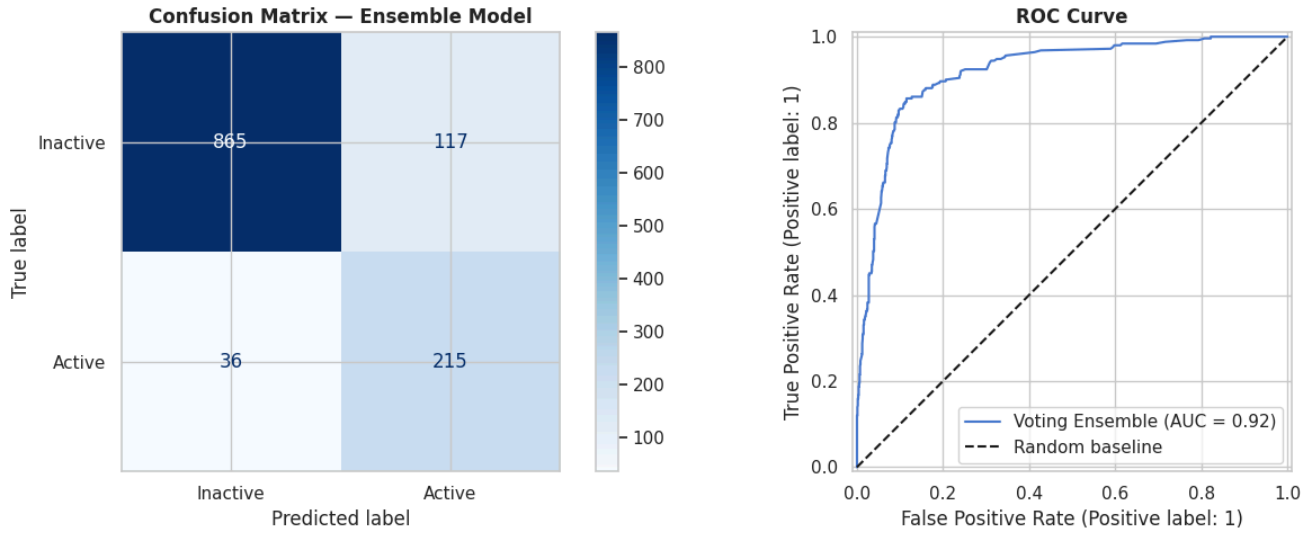
```
Random Forest             AUC: 0.9331 ± 0.0050
Gradient Boosting         AUC: 0.9373 ± 0.0050
Logistic Regression       AUC: 0.9194 ± 0.0051
Voting Ensemble           AUC: 0.9357 ± 0.0050
```



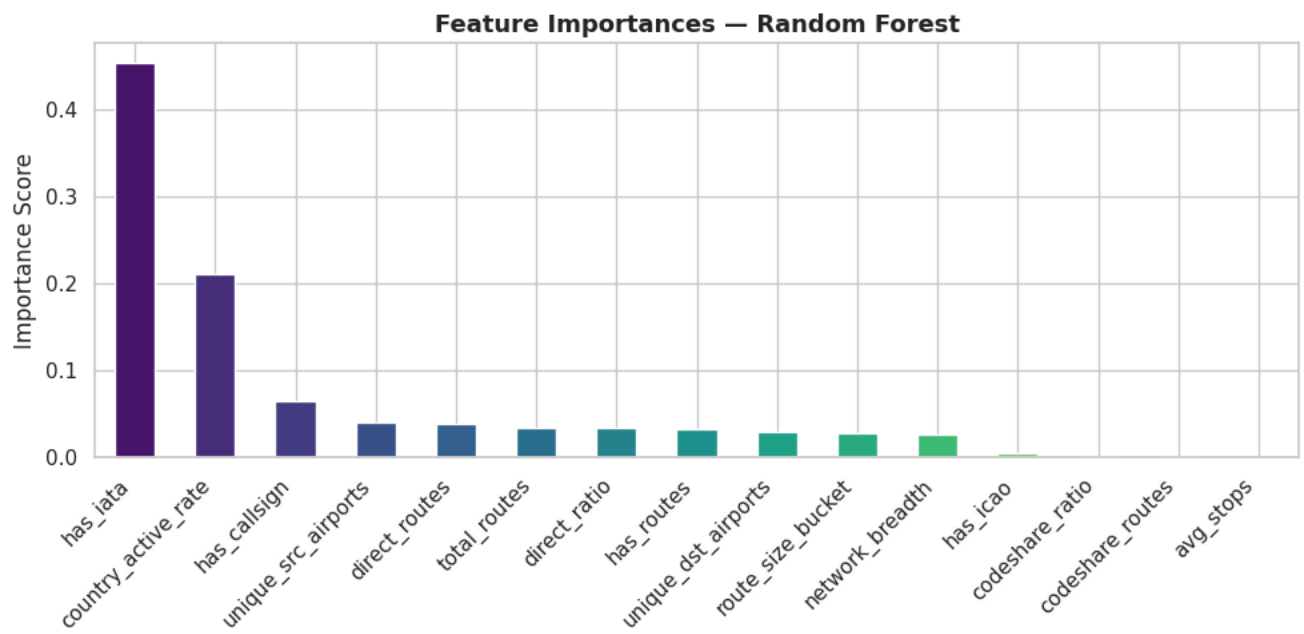

```
# ── 6.5 Train the Voting Ensemble ──────────────────────────────────────
# Note: VotingClassifier fits internal clones of estimators.
# We also fit rf_model separately so it can be used for SHAP later.
ensemble.fit(X_train_res, y_train_res)
rf_model.fit(X_train_res, y_train_res)   # standalone RF for SHAP TreeExplainer

y_pred       = ensemble.predict(X_test)
y_pred_proba = ensemble.predict_proba(X_test)[:, 1]

print('=== TEST SET PERFORMANCE ===')
print(classification_report(y_test, y_pred, target_names=['Inactive', 'Active']))
print(f'ROC-AUC : {roc_auc_score(y_test, y_pred_proba):.4f}')
print(f'Accuracy: {accuracy_score(y_test, y_pred):.4f}')
```

```
=== TEST SET PERFORMANCE ===
              precision    recall  f1-score   support

    Inactive       0.96      0.88      0.92       982
      Active       0.65      0.86      0.74       251

    accuracy                           0.88      1233
   macro avg       0.80      0.87      0.83      1233
weighted avg       0.90      0.88      0.88      1233

ROC-AUC : 0.9205
Accuracy: 0.8759
```

```
# ── 6.6 Confusion Matrix & ROC Curve ───────────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Confusion matrix
ConfusionMatrixDisplay.from_predictions(
    y_test, y_pred,
    display_labels=['Inactive', 'Active'],
    cmap='Blues', ax=axes[0]
)
axes[0].set_title('Confusion Matrix — Ensemble Model', fontsize=12, fontweight='bold')

# ROC curve
RocCurveDisplay.from_predictions(y_test, y_pred_proba, ax=axes[1],
                                 name='Voting Ensemble')
axes[1].plot([0, 1], [0, 1], 'k--', label='Random baseline')
axes[1].set_title('ROC Curve', fontsize=12, fontweight='bold')
axes[1].legend()

plt.tight_layout()
plt.savefig('powerbi_exports/fig_model_evaluation.png', dpi=150, bbox_inches='tight')
plt.show()
```




```
# ── 6.7 Feature Importance (Random Forest) ──────────────────────────────
feat_imp = pd.Series(rf_model.feature_importances_, index=FEATURES)
feat_imp = feat_imp.sort_values(ascending=False)

plt.figure(figsize=(10, 5))
feat_imp.plot(kind='bar', color=sns.color_palette('viridis', len(FEATURES)), edgecolor='white')
plt.title('Feature Importances — Random Forest', fontsize=13, fontweight='bold')
plt.ylabel('Importance Score')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.savefig('powerbi_exports/fig_feature_importance.png', dpi=150, bbox_inches='tight')
plt.show()
```



---

7. Model Explainability (SHAP)

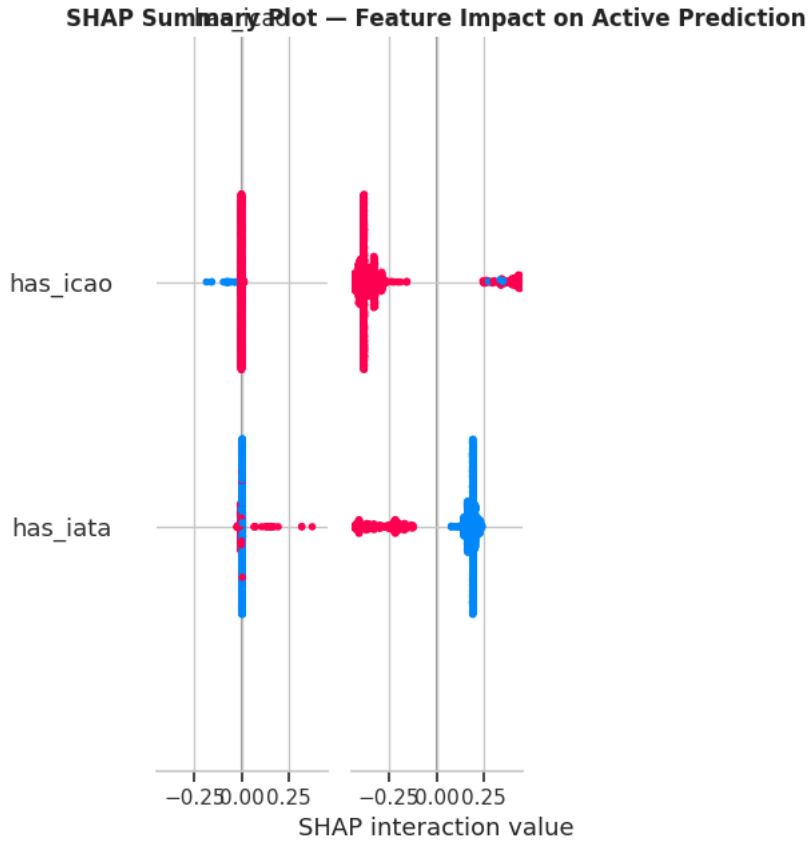
> A high-accuracy model is only useful if stakeholders trust it. SHAP (SHapley Additive exPlanations) opens the black box and shows exactly which features pushed each prediction towards Active or Inactive — at both the global and individual prediction level.

```
# ── 7.1 SHAP TreeExplainer ──────────────────────────────────────────────
# Use the Random Forest component for SHAP (tree-based, fast)
explainer    = shap.TreeExplainer(rf_model)
shap_values  = explainer.shap_values(X_test)

# shap_values is a list of arrays for each class; [1] = active class
shap_active = shap_values[1] if isinstance(shap_values, list) else shap_values

print(f'SHAP values shape: {shap_active.shape}')
```

```
SHAP values shape: (1233, 15, 2)
```

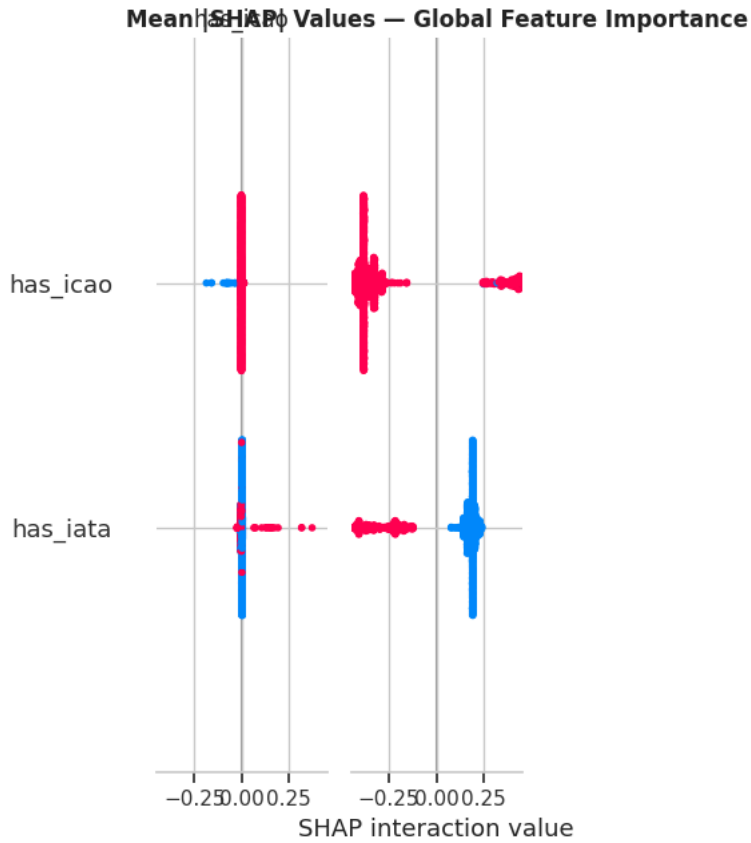
```
# ── 7.2 SHAP Summary Plot (Beeswarm) ────────────────────────────────────
plt.figure(figsize=(10, 7))
shap.summary_plot(shap_active, X_test, feature_names=FEATURES, show=False)
plt.title('SHAP Summary Plot — Feature Impact on Active Prediction', fontsize=12, fontweight='bold')
plt.tight_layout()
plt.savefig('powerbi_exports/fig_shap_summary.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
<Figure size 1000x700 with 0 Axes>
```

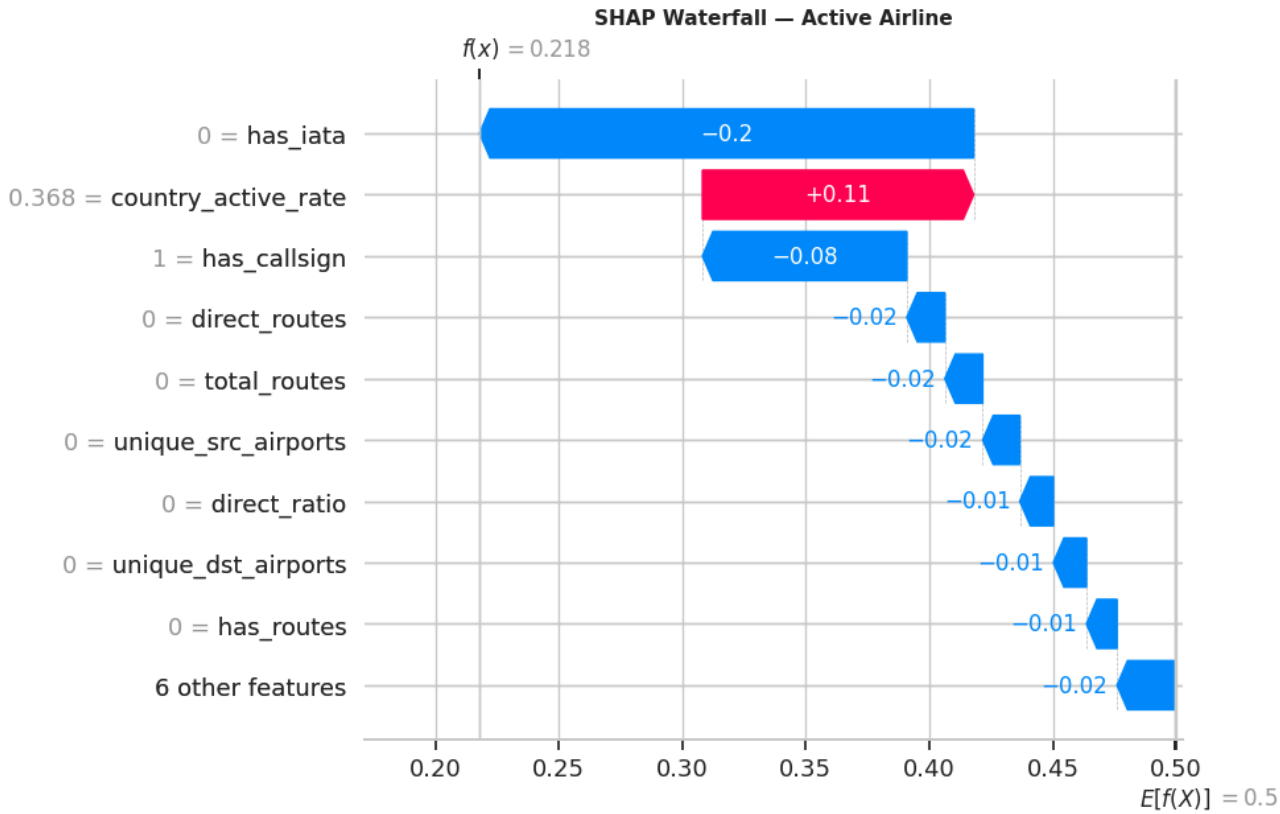



```
# ── 7.3 SHAP Bar Plot (Mean Absolute SHAP values) ──────────────────────
plt.figure(figsize=(10, 5))
shap.summary_plot(shap_active, X_test, feature_names=FEATURES, plot_type='bar', show=False)
plt.title('Mean |SHAP| Values — Global Feature Importance', fontsize=12, fontweight='bold')
plt.tight_layout()
plt.savefig('powerbi_exports/fig_shap_bar.png', dpi=150, bbox_inches='tight')
plt.show()
```

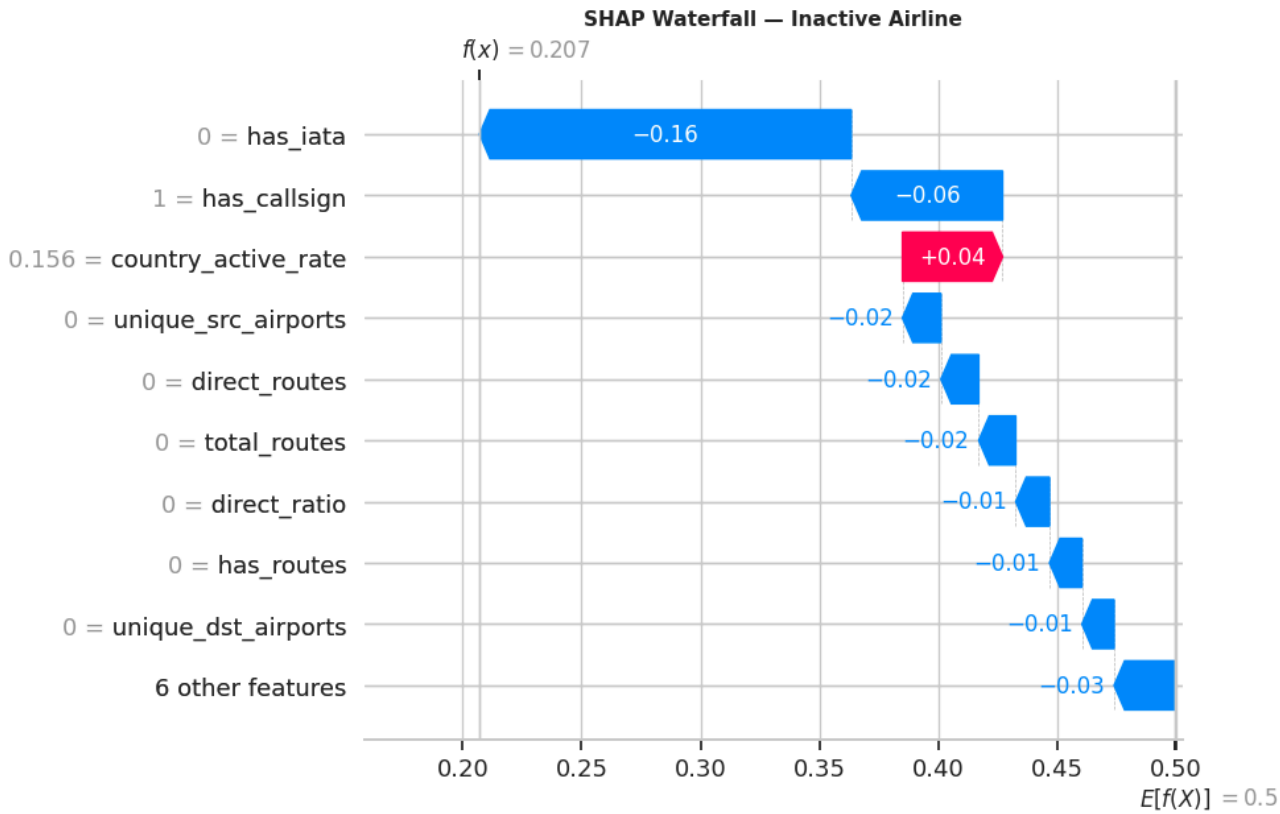
```
<Figure size 1000x500 with 0 Axes>
```




```
# ── 7.4 SHAP Waterfall — individual prediction explanation ──

active_idx = X_test[y_test == 1].index[0]
inactive_idx = X_test[y_test == 0].index[0]

# Fix base value
if isinstance(explainer.expected_value, (list, np.ndarray)):
    base_val = explainer.expected_value[1]
else:
    base_val = explainer.expected_value

for label, idx in [('Active Airline', active_idx), ('Inactive Airline', inactive_idx)]:

    row_pos = X_test.index.get_loc(idx)

    #   FIX HERE
    single_shap = shap_active[row_pos][:, 1] if len(shap_active.shape) == 3 else shap_active[row_pos]

    exp = shap.Explanation(
        values=single_shap,
        base_values=base_val,
        data=X_test.iloc[row_pos].values,
        feature_names=FEATURES
    )

    plt.figure()
    shap.plots.waterfall(exp, show=False)
    plt.title(f'SHAP Waterfall — {label}', fontsize=11, fontweight='bold')

    safe_name = label.lower().replace(' ', '_')
    plt.savefig(f'powerbi_exports/fig_shap_waterfall_{safe_name}.png',
                dpi=150, bbox_inches='tight')

    plt.show()
    print(f' Waterfall saved for {label}')
```




```
Waterfall saved for Active Airline
```




```
Waterfall saved for Inactive Airline
```

8. Bias Analysis & Fairness

> A model that performs well on average but poorly for specific regions could be unfair and misleading. Here we split the test set by country group and measure Precision, Recall, and F1 separately — identifying any systematic disparities in model

> performance across geographic groups.

```
# ── 8.1 Bias definition ──────────────────────────────────────────────────────
# Sensitive attribute: Country (geographic bias)
# We assess whether model performance differs significantly across major country groups

# Add country and predictions to the test set
test_df = X_test.copy()
test_df['true_label']  = y_test.values
test_df['pred_label']  = y_pred
test_df['pred_proba']  = y_pred_proba
test_df['country']     = df.loc[X_test.index, 'Country'].values

# Group countries into regions by active rate
test_df['country_rate_group'] = pd.cut(
    test_df['country_active_rate'],
    bins=[0, 0.33, 0.66, 1.01],
    labels=['Low-rate (<33%)', 'Mid-rate (33-66%)', 'High-rate (>66%)']
)

print('Test set by country rate group:')
display(test_df['country_rate_group'].value_counts())
```

```
Test set by country rate group:
```

country_rate_group	count
Low-rate (<33%)	954
Mid-rate (33-66%)	213
High-rate (>66%)	31

dtype: int64

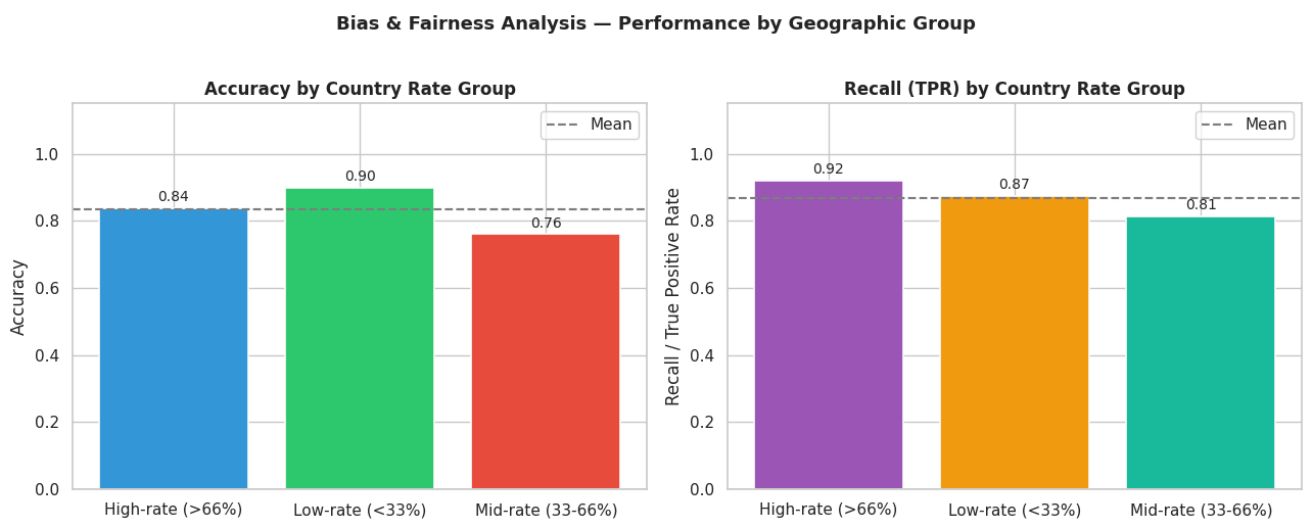
```
# ── 8.2 Performance metrics by group ─────────────────────────────────────────
bias_rows = []
for group in sorted(test_df['country_rate_group'].dropna().unique()):
    sub = test_df[test_df['country_rate_group'] == group]
    if len(sub) < 10:
        continue
    # AUC requires at least 2 classes in the subgroup
    try:
        auc_val = roc_auc_score(sub['true_label'], sub['pred_proba'])
    except ValueError:
        auc_val = np.nan

    bias_rows.append({
        'Group'    : str(group),
        'N'        : len(sub),
        'Accuracy' : round(accuracy_score(sub['true_label'], sub['pred_label']), 4),
        'Precision': round(precision_score(sub['true_label'], sub['pred_label'], zero_division=0), 4),
        'Recall'   : round(recall_score(sub['true_label'], sub['pred_label'], zero_division=0), 4),
        'F1'       : round(f1_score(sub['true_label'], sub['pred_label'], zero_division=0), 4),
        'AUC'      : round(auc_val, 4) if not np.isnan(auc_val) else 'N/A'
    })

bias_df = pd.DataFrame(bias_rows)
print('=== BIAS ANALYSIS — Performance by Country Rate Group ===')
display(bias_df.set_index('Group'))
```

```
=== BIAS ANALYSIS — Performance by Country Rate Group ===
```

Group	N	Accuracy	Precision	Recall	F1	AUC
High-rate (>66%)	31	0.84	0.88	0.92	0.90	0.86
Low-rate (<33%)	954	0.90	0.60	0.87	0.71	0.92
Mid-rate (33-66%)	213	0.76	0.69	0.81	0.74	0.82

```
# ── 8.3 Visualise bias metrics across groups ─────────────────────────────────
if len(bias_df) > 0:
```

```
    fig, axes = plt.subplots(1, 2, figsize=(13, 5))

    # Accuracy by group
    axes[0].bar(bias_df['Group'], bias_df['Accuracy'],
                color=['#3498db','#2ecc71','#e74c3c'][:len(bias_df)],
                edgecolor='white', linewidth=1.2)
    axes[0].axhline(bias_df['Accuracy'].mean(), color='gray', linestyle='--', label='Mean')
    axes[0].set_title('Accuracy by Country Rate Group', fontsize=12, fontweight='bold')
    axes[0].set_ylabel('Accuracy')
    axes[0].set_ylim(0, 1.15)
    axes[0].legend()
    for i, (_, row) in enumerate(bias_df.iterrows()):
        axes[0].text(i, row['Accuracy'] + 0.02, f"{row['Accuracy']:.2f}",
                     ha='center', fontsize=10)

    # Recall (TPR) by group — key fairness metric
    axes[1].bar(bias_df['Group'], bias_df['Recall'],
                color=['#9b59b6','#f39c12','#1abc9c'][:len(bias_df)],
                edgecolor='white', linewidth=1.2)
    axes[1].axhline(bias_df['Recall'].mean(), color='gray', linestyle='--', label='Mean')
    axes[1].set_title('Recall (TPR) by Country Rate Group', fontsize=12, fontweight='bold')
    axes[1].set_ylabel('Recall / True Positive Rate')
    axes[1].set_ylim(0, 1.15)
    axes[1].legend()
    for i, (_, row) in enumerate(bias_df.iterrows()):
        axes[1].text(i, row['Recall'] + 0.02, f"{row['Recall']:.2f}",
                     ha='center', fontsize=10)

    plt.suptitle('Bias & Fairness Analysis — Performance by Geographic Group',
                 fontsize=13, fontweight='bold', y=1.02)
    plt.tight_layout()
    plt.savefig('powerbi_exports/fig_bias_analysis.png', dpi=150, bbox_inches='tight')
    plt.show()
else:
    print('No bias data to plot.')
```




```
# ── 8.4 Equalized Odds Check ──────────────────────────────────────────
print('=== EQUALIZED ODDS CHECK ===')
print('True Positive Rate (Recall) by group:')

# Filter to rows where Recall is numeric
bias_df_numeric = bias_df[bias_df['Recall'].apply(lambda x: isinstance(x, float))]

for _, row in bias_df_numeric.iterrows():
    print(f"  {row['Group']:<25} Recall (TPR) = {row['Recall']:.3f}")

if len(bias_df_numeric) >= 2:
```

```
    tpr_range = bias_df_numeric['Recall'].max() - bias_df_numeric['Recall'].min()
    print(f'\nTPR disparity across groups: {tpr_range:.3f}')
    if tpr_range > 0.10:
        print('  DISPARITY DETECTED (>10%) — Consider threshold adjustment or re-weighting by group.')
    else:
        print(' Equalized Odds satisfied — TPR disparity is within acceptable range (<10%).')
else:
    print('Not enough groups with valid Recall to compute disparity.')

print('\nFairness Mitigation Strategy:')
print('  1. Apply group-specific classification thresholds')
print('  2. Collect more data from underrepresented countries')
print('  3. Use fairness-aware re-weighting during training (sample_weight)')
```

```
=== EQUALIZED ODDS CHECK ===
True Positive Rate (Recall) by group:
  High-rate (>66%)          Recall (TPR) = 0.920
  Low-rate (<33%)           Recall (TPR) = 0.874
  Mid-rate (33-66%)         Recall (TPR) = 0.813

TPR disparity across groups: 0.107
  DISPARITY DETECTED (>10%) — Consider threshold adjustment or re-weighting by group.

Fairness Mitigation Strategy:
  1. Apply group-specific classification thresholds
  2. Collect more data from underrepresented countries
  3. Use fairness-aware re-weighting during training (sample_weight)
```

---

9. Model Packaging & Scoring Script

> A model that only lives inside a notebook has limited real-world value. Here we save the trained ensemble, country lookup table, and metadata to a `model_package/` folder, and generate a standalone `score.py` script so any team can run predictions on new airline data with a single command — no notebook required.

```
# ── 9.1 Save the trained model ──────────────────────────────────────────────
os.makedirs('model_package', exist_ok=True)

# Save the voting ensemble and the standalone RF (used for SHAP)
joblib.dump(ensemble, 'model_package/airline_active_ensemble.pkl')
joblib.dump(rf_model, 'model_package/airline_active_rf.pkl')

# Save the country active rates dict — needed by score.py at inference time
# This is the same dict computed during training (target encoding per country)
joblib.dump(country_active_rate, 'model_package/country_active_rates.pkl')
print(f' country_active_rates.pkl saved ({len(country_active_rate)} countries)')

# Save feature list, metadata, and performance metrics
model_metadata = {
    'model_name'      : 'Airline Active Status Classifier',
    'version'         : '1.0.0',
    'features'        : FEATURES,
    'target'          : TARGET,
    'target_mapping'  : {'0': 'Inactive', '1': 'Active'},
    'training_samples': int(X_train_res.shape[0]),
    'test_auc'        : round(roc_auc_score(y_test, y_pred_proba), 4),
    'test_accuracy'   : round(accuracy_score(y_test, y_pred), 4),
    'algorithm'       : 'Soft Voting Ensemble (RF × 3 + GBM × 2 + LR × 1)',
    'class_balance'   : {'Active (Y)': int(y.sum()), 'Inactive (N)': int((y==0).sum())},
    'description'     : 'Predicts whether an airline is currently active based on route features and regional statistics.'
}

with open('model_package/metadata.json', 'w') as f:
    json.dump(model_metadata, f, indent=2)

print(' Model saved to model_package/')
print(json.dumps(model_metadata, indent=2))
```

```
 country_active_rates.pkl saved (277 countries)
 Model saved to model_package/
{
  "model_name": "Airline Active Status Classifier",
  "version": "1.0.0",
```

```
  "features": [
    "has_iata",
    "has_icao",
    "has_callsign",
    "total_routes",
    "unique_src_airports",
    "unique_dst_airports",
    "codeshare_routes",
    "avg_stops",
    "direct_routes",
    "codeshare_ratio",
    "direct_ratio",
    "network_breadth",
    "country_active_rate",
    "has_routes",
    "route_size_bucket"
  ],
  "target": "Active_binary",
  "target_mapping": {
    "0": "Inactive",
    "1": "Active"
  },
  "training_samples": 7850,
  "test_auc": 0.9205,
  "test_accuracy": 0.8759,
  "algorithm": "Soft Voting Ensemble (RF \u00d7 3 + GBM \u00d7 2 + LR \u00d7 1)",
  "class_balance": {
    "Active (Y)": 1254,
    "Inactive (N)": 4907
  },
  "description": "Predicts whether an airline is currently active based on route features and regional statistics."
}
```


```
# ── 9.2 Scoring script (score.py) ──────────────────────────────────────────
scoring_script = '''
"""
score.py  —  Scoring Script for Airline Active Status Classifier
================================================================
Usage:
    python score.py --input new_airlines.csv --output predictions.csv
    python score.py --input new_airlines.csv --output predictions.csv --routes routes.csv

Input CSV must contain:   Name, IATA, ICAO, Callsign, Country
Optional routes CSV adds: total_routes, codeshare, stops, etc.
"""
import argparse
import pandas as pd
import numpy as np
import joblib
import json
import os

# ── Paths ──────────────────────────────────────────────────────────────────
MODEL_DIR          = os.path.join(os.path.dirname(__file__), 'model_package')
MODEL_PATH         = os.path.join(MODEL_DIR, 'airline_active_ensemble.pkl')
METADATA_PATH      = os.path.join(MODEL_DIR, 'metadata.json')
COUNTRY_RATES_PATH = os.path.join(MODEL_DIR, 'country_active_rates.pkl')


def load_artifacts():
    """Load model, metadata, and country active-rate lookup."""
    model = joblib.load(MODEL_PATH)
    with open(METADATA_PATH) as f:
        metadata = json.load(f)

    country_rates = joblib.load(COUNTRY_RATES_PATH)
    global_mean = np.mean(list(country_rates.values()))

    print(f"Loaded: {metadata['model_name']} v{metadata['version']}")
    print(f"Features: {metadata['features']}")
    print(f"Countries in lookup: {len(country_rates)}, global mean rate: {global_mean:.3f}")
    return model, metadata, country_rates, global_mean


def engineer_features(airlines_df, routes_df, country_rates, global_mean):
    """
    Replicate the same feature engineering used during training.
    Uses saved country_active_rates from training for consistent target encoding.
    """
```

```
    df = airlines_df.copy()

    # ── Clean identifiers ──────────────────────────────────────────────────
    df['IATA'] = df['IATA'].replace(['-', '\\N', 'nan', ''], np.nan)
    df['ICAO'] = df.get('ICAO', pd.Series(index=df.index, dtype=str)).replace(['-', '\\N', 'nan', ''], np.nan)
    df['Callsign'] = df.get('Callsign', pd.Series(index=df.index, dtype=str)).replace(['-', '\\N', 'nan', ''], np.nan)
    df['Country'] = df['Country'].astype(str).str.strip().str.title()

    # ── Route aggregation ──────────────────────────────────────────────────
    if routes_df is not None:
        route_agg = routes_df.groupby('Airline').agg(
            total_routes=('Airline', 'count'),
            unique_src_airports=('Source Airport', 'nunique'),
            unique_dst_airports=('Destination Airport', 'nunique'),
            codeshare_routes=('Codeshare', lambda x: (x == 'Y').sum()),
            avg_stops=('Stops', 'mean'),
            direct_routes=('Stops', lambda x: (x == 0).sum())
        ).reset_index().rename(columns={'Airline': 'IATA'})

        route_agg['codeshare_ratio'] = route_agg['codeshare_routes'] / route_agg['total_routes']
        route_agg['direct_ratio'] = route_agg['direct_routes'] / route_agg['total_routes']
        route_agg['network_breadth'] = route_agg['unique_src_airports'] + route_agg['unique_dst_airports']

        df = df.merge(route_agg, on='IATA', how='left')

    # Fill missing route columns with 0
    route_cols = [
        'total_routes', 'unique_src_airports', 'unique_dst_airports',
        'codeshare_routes', 'avg_stops', 'direct_routes',
        'codeshare_ratio', 'direct_ratio', 'network_breadth'
    ]
    for col in route_cols:
        if col not in df.columns:
            df[col] = 0
    df[route_cols] = df[route_cols].fillna(0)

    # ── Indicator features ─────────────────────────────────────────────────
    df['has_iata'] = (~df['IATA'].isnull()).astype(int)
    df['has_icao'] = (~df.get('ICAO', pd.Series(np.nan, index=df.index)).isnull()).astype(int)
    df['has_callsign'] = (~df.get('Callsign', pd.Series(np.nan, index=df.index)).isnull()).astype(int)
    df['has_routes'] = (df['total_routes'] > 0).astype(int)

    # ── Country active rate ────────────────────────────────────────────────
    df['country_active_rate'] = df['Country'].map(country_rates).fillna(global_mean)

    # ── Route size bucket ──────────────────────────────────────────────────
    df['route_size_bucket'] = pd.cut(
        df['total_routes'],
        bins=[-1, 0, 10, 50, 200, 10000],
        labels=[0, 1, 2, 3, 4]
    ).astype(float).fillna(0).astype(int)

    return df


def score(input_path, output_path, routes_path=None):
    """Score new airlines and save predictions to CSV."""
    model, metadata, country_rates, global_mean = load_artifacts()
    features = metadata['features']

    airlines_df = pd.read_csv(input_path)
    routes_df = pd.read_csv(routes_path) if routes_path else None
    print(f"\\nScoring {len(airlines_df):,} airlines...")

    df = engineer_features(airlines_df, routes_df, country_rates, global_mean)

    X = df[features].fillna(0)
    probabilities = model.predict_proba(X)[:, 1]
    predictions = (probabilities >= 0.5).astype(int)

    out_cols = [c for c in ['Name', 'IATA', 'Country'] if c in airlines_df.columns]
    output_df = airlines_df[out_cols].copy()
    output_df['predicted_active'] = predictions
    output_df['predicted_active_label'] = output_df['predicted_active'].map({1: 'Active', 0: 'Inactive'})
    output_df['confidence'] = probabilities.round(4)

    output_df.to_csv(output_path, index=False)
```

```
    print(f" Predictions saved → {output_path}")
    print(f"   Active: {predictions.sum():,} | Inactive: {(predictions == 0).sum():,}")
    print(output_df.head(10).to_string(index=False))
    return output_df


if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Score airlines with trained ensemble model.')
    parser.add_argument('--input', required=True, help='Input CSV path (must have Name, IATA, Country columns)')
    parser.add_argument('--output', required=True, help='Output predictions CSV path')
    parser.add_argument('--routes', default=None, help='(Optional) Routes CSV for richer features')
    args = parser.parse_args()
    score(args.input, args.output, args.routes)
'''

with open('model_package/score.py', 'w') as f:
    f.write(scoring_script.strip())

print(' score.py saved to model_package/score.py')
print('   country_active_rate is now loaded from model_package/country_active_rates.pkl')
print('   unseen countries fall back to the global training mean (not hardcoded 0.5)')
```

```
 score.py saved to model_package/score.py
   country_active_rate is now loaded from model_package/country_active_rates.pkl
   unseen countries fall back to the global training mean (not hardcoded 0.5)
```

```
# ── 9.3 Verify package contents ──────────────────────────────────────────
import glob
files = glob.glob('model_package/*')
print('model_package/ contents:')
for f in sorted(files):
    size = os.path.getsize(f)
    print(f'  {os.path.basename(f):<45} {size/1024:.1f} KB')
```

```
model_package/ contents:
  airline_active_ensemble.pkl                   12501.7 KB
  airline_active_rf.pkl                         5844.6 KB
  country_active_rates.pkl                      5.8 KB
  metadata.json                                 0.8 KB
  score.py                                      6.6 KB
```

---

10. Power BI Data Exports

> The model predictions and enriched datasets are exported as CSV files for direct loading into Power BI. All analysis charts are also saved as PNG files for use in the dashboard, reports, and presentations.

```
# ── 10. Power BI Data Exports ────────────────────────────────────────────
os.makedirs('powerbi_exports', exist_ok=True)

# Export 1: Enriched airlines (main fact table for Power BI)
export_cols = ['Name','IATA','ICAO','Callsign','Country','Active',
               'total_routes','unique_src_airports','unique_dst_airports',
               'codeshare_ratio','direct_ratio','network_breadth',
               'country_active_rate','has_routes','Active_binary']
airlines_export = df[export_cols].copy()
airlines_export['pred_active']      = ensemble.predict(X)
airlines_export['pred_probability'] = ensemble.predict_proba(X)[:,1].round(4)
airlines_export.to_csv('powerbi_exports/airlines_enriched.csv', index=False)
print(f' airlines_enriched.csv       — {len(airlines_export):,} rows')

# Export 2: Country summary
country_summary = df.groupby('Country').agg(
    total_airlines   = ('Active_binary', 'count'),
    active_airlines  = ('Active_binary', 'sum'),
    active_rate      = ('Active_binary', 'mean'),
    avg_routes       = ('total_routes', 'mean'),
    total_routes     = ('total_routes', 'sum')
).reset_index().round(3)
country_summary['inactive_airlines'] = country_summary['total_airlines'] - country_summary['active_airlines']
country_summary.to_csv('powerbi_exports/country_summary.csv', index=False)
print(f' country_summary.csv         — {len(country_summary):,} rows')

# Export 3: Routes with lat/lon for map visuals
```

```
routes_export = routes.merge(
    airports[['IATA','City','Country','Latitude','Longitude']],
    left_on='Source Airport', right_on='IATA', how='left'
).rename(columns={'City':'src_city','Country':'src_country',
                  'Latitude':'src_lat','Longitude':'src_lon'}).drop(columns='IATA', errors='ignore')
routes_export = routes_export.merge(
    airports[['IATA','City','Country','Latitude','Longitude']],
    left_on='Destination Airport', right_on='IATA', how='left'
).rename(columns={'City':'dst_city','Country':'dst_country',
                  'Latitude':'dst_lat','Longitude':'dst_lon'}).drop(columns='IATA', errors='ignore')
routes_export.to_csv('powerbi_exports/routes_enriched.csv', index=False)
print(f' routes_enriched.csv         — {len(routes_export):,} rows')
```